



COMP1006 - WEEK FIVE

BOOTSTRAP, CREATE & READ

TODAY'S SCHEDULE

1. Check In
2. Review Form Validation
3. A Little Bit of Bootstrap
4. Review Connecting to a Database
5. Building Create & Read Functionality
6. Lab Four Intro & Work Period
7. Summary, Task List, Next Week

```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
```


LEARNING OBJECTIVES

1. Capture user data from an HTML form
2. Validate user supplied data using client-side and server-side techniques
3. Utilize Bootstrap web framework for UI design
4. Store & retrieve data from a MySQL database

```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
```

PART ONE : CHECK IN

CODING

...
In progress

WEEK FIVE CHECK IN

[JOINMYQUIZ.COM](https://joinmyquiz.com)



Image Source: pexels.com

PART TWO: WORKING WITH FORMS AND FORM DATA



WHAT IS SANITIZING DATA?

WHY DO WE DO IT?

HOW DO WE DO IT?


```
//grab the data, clean and store in a variable - santize!  
$firstName = filter_input(INPUT_POST, 'first_name', FILTER_SANITIZE_SPECIAL_CHARS);  
$lastName = filter_input(INPUT_POST, 'last_name', FILTER_SANITIZE_SPECIAL_CHARS);  
$address = filter_input(INPUT_POST, 'address', FILTER_SANITIZE_SPECIAL_CHARS);  
$email = filter_input(INPUT_POST, 'email', FILTER_SANITIZE_EMAIL);
```

HOW DO WE ENSURE WE GET
THE RIGHT DATA?

```
|  
//require email and validate proper format  
if ($email === null || $email === '') {  
    $errors[] = "Email is Required";  
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {  
    $errors[] = "Email must be a valid email";  
}
```


PART THREE: A LITTLE BIT OF BOOTSTRAP



Better Way to Track Our Orders?



Lisa Dough <lisa@bakeittillyoumakeit.com>

Today, 10:17 AM

Hi Team,

As you know, it's our busy season and we're getting lots of customer orders coming through via email.

I was wondering if you could help me find a better way to store and track our orders? 👍

Trying to keep track of everything this way is getting overwhelming.

Do you think there's a more organized solution? 😊

Thanks again, Lisa

Lisa Dough

[Bake It Till You Make It Bakery](#) 🏠

↩ Reply

➦ Forward

HOW COULD YOU **IMPROVE** THE LOOK OF THIS
FORM **WITHOUT IT COSTING THE CLIENT A LOT**
OF \$\$\$

A LIL' BIT ABOUT BOOTSTRAP



- ▶ a **framework** for building responsive, mobile-first websites & applications
- ▶ Excellent for **rapid development**
- ▶ **Easy** to use & integrate
- ▶ Some **drawbacks**

BOOTSTRAP DEMO

PART FOUR: CONNECTING TO A DATABASE



LET'S REVISIT **CONNECT.PHP**

PHP Connections with PDO



PART FIVE: CREATE & READ

CODING

...
In progress

Better Way to Track Our Orders?



Lisa Dough <lisa@bakeittillyoumakeit.com>

Today, 10:17 AM

Hi Team,

As you know, it's our busy season and we're getting lots of customer orders coming through via email.

I was wondering if you could help me find a better way to store and track our orders? 👍

Trying to keep track of everything this way is getting overwhelming.

Do you think there's a more organized solution? 😊

Thanks again, Lisa

Lisa Dough

[Bake It Till You Make It Bakery](#) 🏠

↩ Reply

➡ Forward

HOW WEB APPS TALK TO THE DATABASE

Typical Workflow:

- ▶ User submits a form
- ▶ PHP receives the user input
- ▶ PHP builds an SQL query
- ▶ Database executes the query
- ▶ Data is stored or returned

**** if handled incorrectly, user input can change how SQL behaves**

CREATE/INSERT

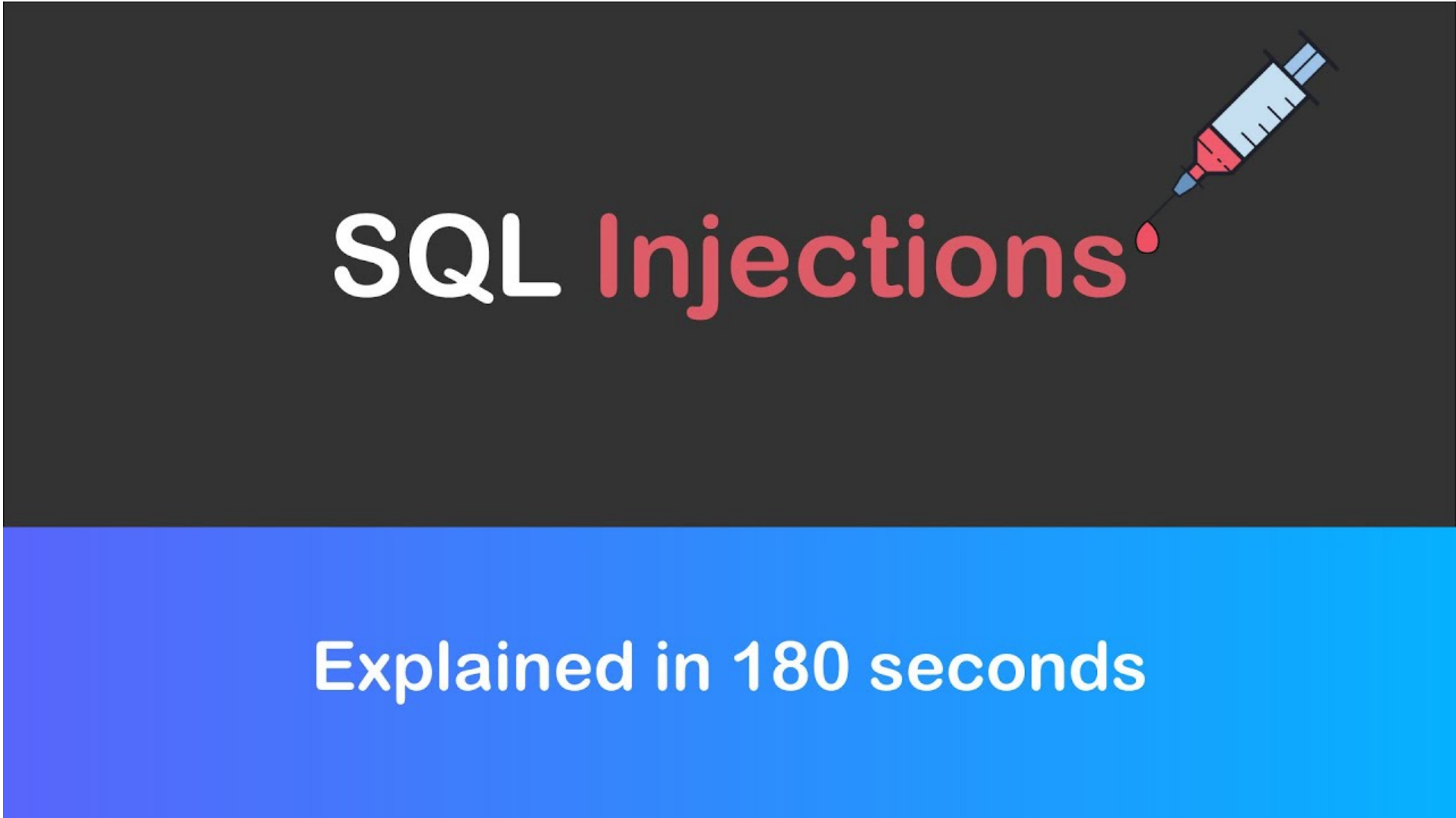
- ▶ Once you **connect to a database**, you can use the connection to execute **SELECT** statements that retrieve data and **INSERT UPDATE** and **DELETE**
- ▶ can use **prepared statements** or dynamic statements

DYNAMIC STATEMENT – WHAT COULD GO WRONG?

php

```
$sql = "SELECT * FROM users WHERE email = '$email'";
```


WHAT IS AN SQL INJECTION?



SQL Injections

Explained in 180 seconds

ISN'T SANITIZATION ENOUGH?



USE PREPARED STATEMENTS

- ▶ Separate **SQL structure** from **data**
- ▶ Prevent **user input from changing query logic**
- ▶ Are built into **PDO**
- ▶ Works by sending SQL structure to the database and user data is sent separately
- ▶ User input is **treated as data only**
- ▶ Prepared statements are the **industry standard**

HOW TO USE PREPARED STATEMENTS

1. Write SQL with **placeholders**
2. **Prepare** the statement
3. **Execute** with **values**

PLACEHOLDERS

```
INSERT INTO songs (title, genre)
VALUES (:title, :genre)
```

- ▶ Act like **variables** in SQL
- ▶ Are **replaced safely** at **execution time**

PREPARE THE STATEMENT

```
$stmt = $pdo->prepare($sql);
```

- ▶ Sends **SQL structure** to the database
- ▶ Database **parses** and **validates** the query
- ▶ No user data is sent **yet**

EXECUTE THE DATA

```
$stmt->execute([  
    ":title" => $title,  
    ":genre" => $genre  
]);
```

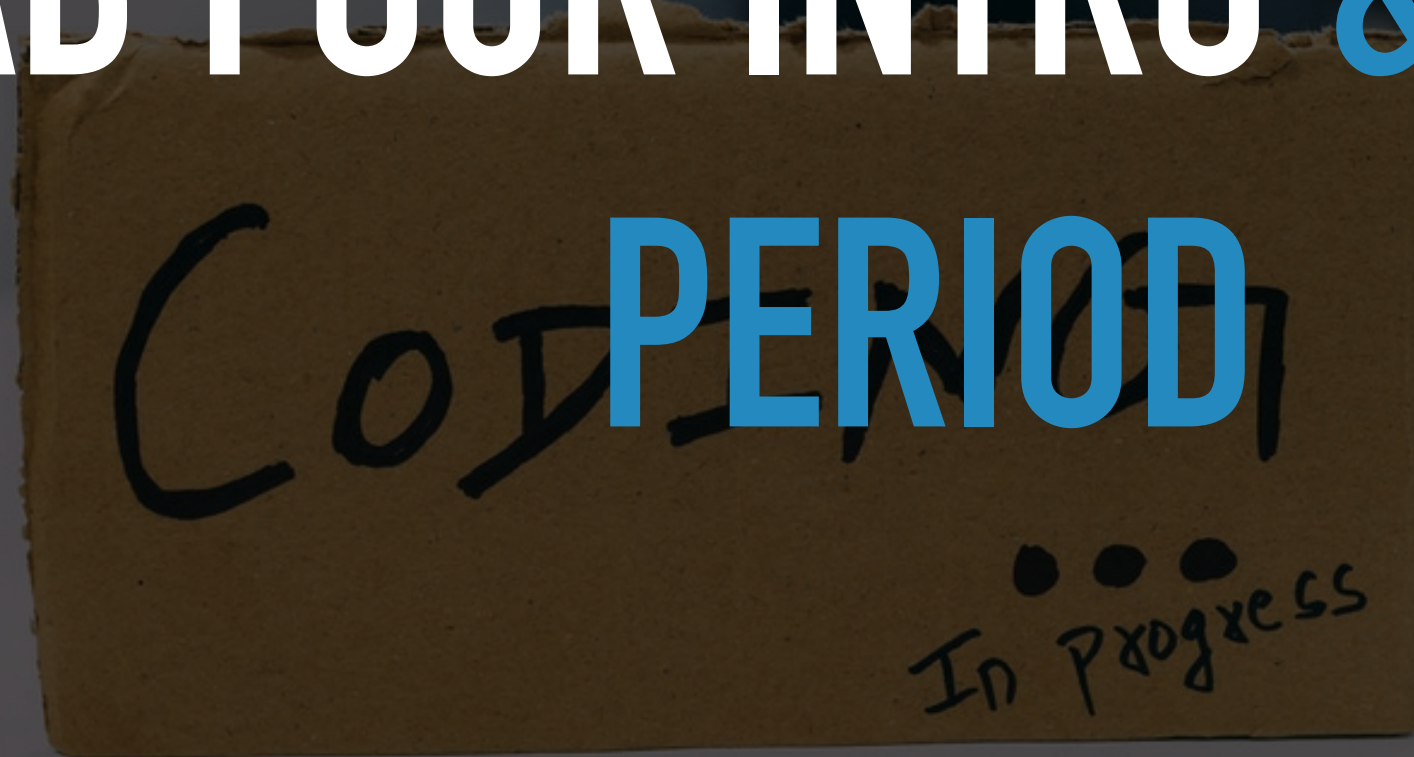
- ▶ Sends user data to the database
- ▶ Safely inserts values into the placeholders
- ▶ Runs the SQL query

LET'S BUILD THIS TOGETHER

READ ...

- ▶ If the SQL statement is a **SELECT statement** that returns a result set, the execute method stores the result set in the PDOStatement object.
- ▶ use the **fetchAll() method** of a PDOStatement object to get an array of **all** of the remaining rows in the result set

LAB FOUR INTRO & WORK PERIOD



COMP 1006 - Winter 2026

LAB FOUR – CREATE & READ

To Complete This Lab:

A client has asked you to help them set up a simple email subscription feature for their website so they can start building a mailing list. They already have a basic subscription form, but they need the backend functionality to store subscriber information in a database and view a list of subscribers. You have been given starter code that includes TODO comments to guide you through the process. Your task is to complete the missing code using PDO prepared statements to insert subscriber data into the database and retrieve it for display.

Learning Objectives:

- design, program, and implement Web pages that interact with Web-enabled databases performing simple CRUD (Create, Read, Update, Delete) operations;

SUMMARY, TO DO

& NEXT WEEK



SUMMARY

1. Review Data Sanitization and Validation
2. A Little Bit of Bootstrap
3. Create and Read

```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
43   if (req.body) {
44     return qs.parse(req.body);
45   } else {
46     throw new Error('Cannot get an OAuth request token');
47   }
48 }
49
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
58   if (req.body) {
59     return qs.parse(req.body);
60   } else {
61     throw new Error('Cannot get an OAuth request token');
62   }
63 }
```

TO DO **THIS WEEK** :

LAB FOUR:

SECTION 01 – FRIDAY, FEB 6TH @ 11:59 PM

SECTION 02 – SUNDAY, FEB 8TH @ 11:59 PM

CONTINUE WORKING ON COURSE PROJECT PHASE ONE – DUE WEEK 6

NEXT WEEK :

**UPDATE & DELETE / MIDTERM
PREP**